
Workgroup:	Network Working Group
Internet-Draft:	draft-wullink-rpp-multiparty-authorization-00
Published:	14 November 2026
Intended Status:	Standards Track
Expires:	18 May 2027
Authors:	M. Wullink P. Kowalik SIDN Labs DENIC

Multi-Party Authorization for RESTful Provisioning Protocol (RPP)

Abstract

The traditional Registrar, Registry, and Registrant model for domain name management is evolving to include third-party service providers, such as DNS hosting providers. This document describes a generic multi-party authorization flow that allows registries to securely delegate domain management functions to accredited third-party service providers, while ensuring that the registrant's explicit consent is obtained before any RPP operations, described in [I-D.ietf-rpp-core], are executed.

TODO

Status of This Memo

This Internet-Draft is submitted in full conformance with the provisions of BCP 78 and BCP 79.

Internet-Drafts are working documents of the Internet Engineering Task Force (IETF). Note that other groups may also distribute working documents as Internet-Drafts. The list of current Internet-Drafts is at <https://datatracker.ietf.org/drafts/current/>.

Internet-Drafts are draft documents valid for a maximum of six months and may be updated, replaced, or obsoleted by other documents at any time. It is inappropriate to use Internet-Drafts as reference material or to cite them other than as "work in progress."

This Internet-Draft will expire on 18 May 2027.

Copyright Notice

Copyright (c) 2026 IETF Trust and the persons identified as the document authors. All rights reserved.

This document is subject to BCP 78 and the IETF Trust's Legal Provisions Relating to IETF Documents (<https://trustee.ietf.org/license-info>) in effect on the date of publication of this document. Please review these documents carefully, as they describe your rights and restrictions with respect to this document. Code Components extracted from this document must include Revised BSD License text as described in Section 4.e of the Trust Legal Provisions and are provided without warranty as described in the Revised BSD License.

Table of Contents

1. Introduction	3
2. Terminology	3
3. Conventions Used in This Document	3
4. Architectural Overview	3
5. HTTP	6
5.1. Endpoints	6
5.2. Approval Header	7
6. Public Keys	7
7. Approval Redirect URLs	8
8. Transaction signing and verification	9
8.1. Canonicalization and Signing Input	9
8.2. Registry	9
8.3. Registrar	10
9. Transaction Types	10
9.1. DNS Update Transaction Type	10
9.2. Transfer Transaction Type	10
9.3. JSON Schema	11
10. Examples	11
10.1. Key provisioning	11
10.2. Domain Update Request	12
10.3. Domain Transfer Request	13
11. Security Considerations	15
12. IANA Considerations	15
13. Internationalization Considerations	16
14. Privacy Considerations	16
15. Change History	16
15.1. Version 00	16

16. References	16
16.1. Normative References	16
16.2. Informative References	17
Acknowledgements	17
Authors' Addresses	17

1. Introduction

The generic multi-party authorization flow described in this document allows registries to securely delegate RPP object operations to accredited third-party service providers, while ensuring that the registrant's explicit consent is obtained before any RPP operations, described in [\[I-D.ietf-rpp-core\]](#), are executed. The registry and the 3rd party MUST have a pre-established trust relationship, how this trust is established is out of scope for this document. The 3rd party does not have a direct trust relationship with the registrar, but the registrar trusts the registry to only accept requests from accredited 3rd parties.

TODO

2. Terminology

In this document the following terminology is used.

URL - A Uniform Resource Locator as defined in [\[RFC3986\]](#).

Resource - An object having a type, data, and possible relationship to other resources, identified by a URL.

RPP client - An HTTP user agent performing an RPP request

RPP server - An HTTP server responsible for processing requests and returning results in any supported media type.

3rd party - A service provider that is accredited by the registry to perform domain management operations on behalf of the registrant.

3. Conventions Used in This Document

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in [\[RFC2119\]](#).

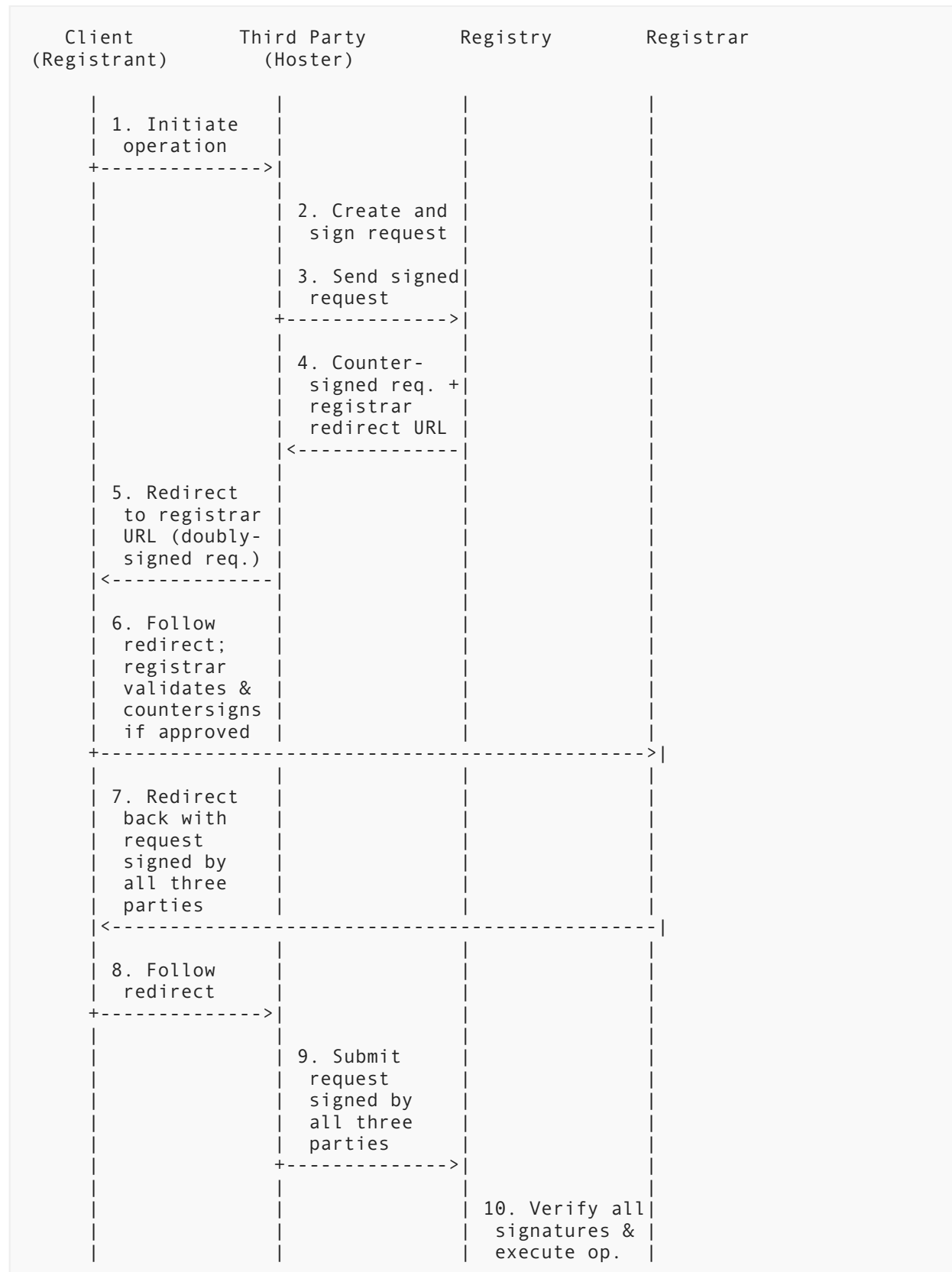
In examples, indentation and white space are provided only to illustrate element relationships and are not REQUIRED features of the protocol.

4. Architectural Overview

RPP enables registries to securely delegate domain management functions to accredited third-party service providers, starting with DNS hosting providers.

The authorization of such a delegated operation requires the explicit participation of four parties: the registrant's client, the third party performing the operation, the registry, and the registrar of record. Each of the three service providers (third party, registry, and registrar) applies its own digital signature to the authorization request, so that the operation can only be executed once all three signatures have been collected and verified. This prevents any single party from unilaterally authorizing a sensitive domain management operation.

The following diagram illustrates the multi-party authorization flow:



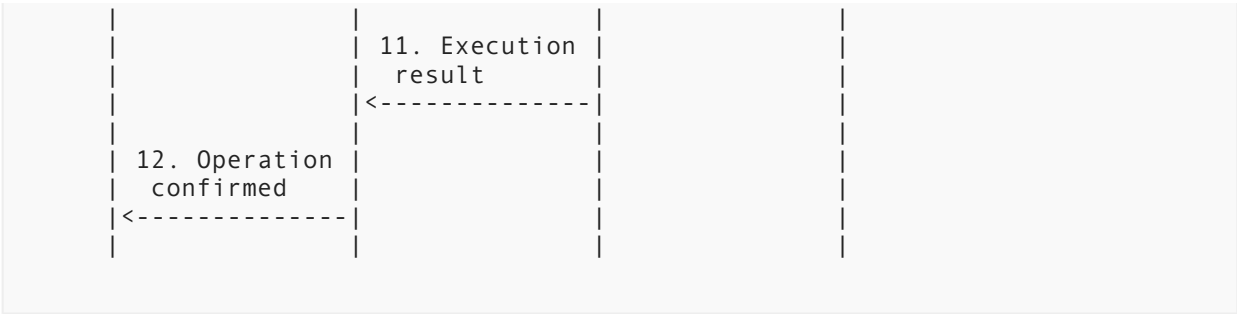


Figure 1: multi-party Authorization Flow

The steps in the diagram are as follows:

1. The client connects to the third party to initiate a domain management operation that requires authorization from the registry and the registrar.
2. The third party's backend constructs a request describing the requested operation and signs it, expressing the third party's intent to perform the operation on behalf of the registrant.
3. The third party sends the signed request to the registry over a direct, back-channel connection.
4. The registry validates the third party's signature and, provided the third party is accredited to act on the object, adds its own signature. The registry returns the request, now signed by both the third party and the registry, together with the URL of the registrar's web-based consent screen to which the third party must redirect the client's browser for transaction approval.
5. The third party redirects the client's browser to the registrar's consent screen at the URL provided by the registry, conveying the doubly-signed request as a parameter of the redirect.
6. The client's browser follows the redirect to the registrar. The registrar validates the signatures of both the third party and the registry, and presents the requested operation to the registrant for approval. If the registrant approves, the registrar adds its own signature, producing a request signed by all three parties.
7. The registrar redirects the client's browser back to the third party, conveying the request signed by all three parties as a parameter of the redirect.
8. The client's browser follows the redirect, delivering the request signed by all three parties to the third party.
9. The third party submits the request signed by all three parties to the registry over a direct, back-channel connection, to request execution of the operation.
10. The registry verifies the signatures of the third party, the registry itself, and the registrar. The registry executes the requested operation only if all three signatures are present and valid.
11. The registry returns the result of the operation to the third party.
12. The third party informs the client that the requested operation has been completed.

5. HTTP

5.1. Endpoints

The following non normative table lists RPP endpoints related to authorisation processes, each derived by applying the rules defined in section "HTTP Mapping Rules" in [I-D.ietf-rpp-core].

Operation	HTTP Method	URL path
Authorisation: create	"POST"	"/{collection}/{id}/processes/authorisationProcesses"
Authorisation: read	"GET"	"/{collection}/{id}/processes/authorisationProcesses/latest"
Authorisation: read	"GET"	"/{collection}/{id}/processes/authorisationProcesses/{id}"
Authorisation: list	"GET"	"/{collection}/{id}/processes/authorisationProcesses"

Table 1

TODO

POST: Create a new authorization request, signed by the registry

5.2. Approval Header

The value of the RPP-Approval header is a base64 encoded string that represents the signed approval request, which is used by the registry to verify that the request has been approved by the registrar and registrant. The header field uses the following ABNF syntax:

```
Approval = "RPP-Approval" ":" SP approval-token
approval-token = 1*CHAR
```

Example HTTP request using the RPP-Approval header in a domain update operation, to update the DNS records for a domain object:

```
PATCH /domainNames/test1.example HTTP/1.1
RPP-Approval: <approval-token>
Authorization: Bearer <access-token>
Content-Type: application/json

**TODO** messagebody here
```

TODO do we want to base64 encode the approval token? It is a JSON object, so it can contain characters that are not allowed in HTTP headers. Base64 encoding would make it safe to include in the header, but it would also make it larger. We could also use a query parameter instead of a header, but that would make it visible in logs and browser history.

6. Public Keys

A registrar that wants to participate in multi-party authorization **MUST** provide at least one public key to the registry. Each key is identified by a unique key identifier, which is used to verify signatures applied by the corresponding private key.

The RPP server MUST provide the current list of public keys and their identifiers to the 3rd party and the registrar, using the discovery document available at the well-known endpoint as described in [I-D.ietf-rpp-core]. The Discovery document MUST be extended to include the additional public keys and their identifiers, using the field name `registryKeys`, which is an array of JSON Web Key (JWK) objects [RFC7517], the `kid` field of each JWK object MUST be used as the key identifier.

The `use` field of each JWK object MUST be set to `sig`, indicating that the key is used for digital signatures. The `alg` field of each JWK object MUST be set to the algorithm used to sign requests, such as RS256 for RSA signatures using SHA-256.

The following example shows a discovery document with 2 registry public keys:

```
{
  "registryKeys": [
    {
      "kty": "RSA",
      "kid": "registry-key-1",
      "use": "sig",
      "alg": "RS256",
      "n": "0vx7agoebGcQSuuPiLJXZptN9nndrQmbXEps2aiAFbWhM78LhWx4cbb...",
      "e": "AQAB"
    },
    {
      "kty": "EC",
      "kid": "registry-key-2",
      "use": "sig",
      "alg": "ES256",
      "crv": "P-256",
      "x": "f830J3D2xF1Bg8vub9tLe1gHMzV76e8Tus9uPHvRVEU",
      "y": "x_FEzRu9m36HLN_tue659LNpXW6pCyStikYjKIWI5a0"
    }
  ]
}
```

7. Approval Redirect URLs

A registrar that supports multi-party authorization MUST provide a web-based consent screen to which the 3rd party can redirect the client's browser to obtain the registrant's approval for the requested operation. The registrar MUST provide a URL for this consent screen to the registry, which is used by the registry to inform the 3rd party where to redirect the client's browser.

This specification extends the Organization type specified in [I-D.ietf-rpp-data-objects] to include an additional field `approvalRedirectUrl`, which is a URL that points to the registrar's web-based consent screen. The `approvalRedirectUrl` field is a string that MUST be a valid URL, and it MUST use the HTTPS scheme.

TODO:

8. Transaction signing and verification

This section describes how each party involved in the multi-party authorization flow must cryptographically sign, and verify the authenticity and integrity of the transaction request. All parties involved in the multi-party authorization flow MUST use strong cryptography to sign and verify requests. The specific algorithms and key management practices are outside the scope of this document, but it is RECOMMENDED that parties follow best practices for cryptographic security.

Each party MUST sign only the data that it is responsible for, and MUST verify the signatures of all other parties involved in the transaction. Each party MUST include its public key identifier in the request or response, so that other parties can verify the signature.

8.1. Canonicalization and Signing Input

Signers MUST NOT construct the signature input by naively concatenating field values as strings. Ad hoc concatenation is ambiguous: for example, concatenating the values "ab" and "c" produces the same string as concatenating "a" and "bc", and differences in whitespace, member ordering, or numeric formatting between a signer's and a verifier's JSON serializer can cause a correctly-signed request to fail verification.

Instead, the signature input MUST be derived by applying the JSON Canonicalization Scheme (JCS) [\[RFC8785\]](#) to a JSON object containing exactly the fields covered by that signature, encoded as UTF-8 octets. JCS produces a single, deterministic byte string for a given set of field values, independent of the original member order or formatting, making the signature input reproducible by any conforming verifier.

Each signature MUST also cover every signature and key identifier field already present in the request at the time that signature is applied. This chains each countersignature to the ones that precede it, so that removing, reordering, or substituting an earlier signature invalidates every signature applied after it. Concretely, for the fields defined in [Section 10](#):

- The registry signs the JCS-canonicalized form of {transactionType, id, timestamp, expiration, objectId, requestorId, data, requestorSig, requestorKeyId, registryKeyId} and places the result in registrySig.
- The registrar signs the JCS-canonicalized form of {transactionType, id, timestamp, expiration, objectId, requestorId, data, requestorSig, requestorKeyId, registrySig, registryKeyId, registrarKeyId} and places the result in registrarSig.

Verifiers MUST validate signatures in the same order they were applied (3rd party, then registry, then registrar), reconstructing the corresponding canonical input at each step, and MUST reject the request if any signature does not verify against its expected canonical input.

8.2. Registry

The registry MUST verify the transaction request, and copy the request to a response document while adding additional transaction related properties, and finally sign the request using its private key. The signature MUST be included in the response object. The registry MUST also include its public key identifier in the response, so that the registrar can verify the signature.

8.3. Registrar

The registrar MUST verify the signature of the registry using the public key linked to the public key identifier in the response. The registrar MUST copy the JSON request into a JSON response object, add additional transactional properties such as the approval status, and sign it using its private key. The signature MUST be included in the response object. The registrar MUST also include its public key identifier in the response, so that the registry can verify the signature.

9. Transaction Types

TODO the transaction types are defined in separate documents, this document only describes the generic flow and the signing and verification of transactions. The transaction types are defined in separate documents, which MUST be registered with IANA as described in [Section 12](#).

This document defines two transaction types for multi-party authorization: `rpp:transaction:dns-update` and `rpp:transaction:transfer`. A transaction type allows access to specific RPP operations for specific objects only. Future specifications may define additional transaction types, which MUST be registered with IANA as described in [Section 12](#).

This specification does not define a bespoke payload format for the operation being authorized. Instead, the data property of a transaction request MUST contain the same JSON request body that would be sent directly to the RPP server for the corresponding operation, using the mapping rules and object representations defined in [\[I-D.ietf-rpp-json\]](#). Reusing the RPP JSON request body avoids maintaining a second, parallel representation of every RPP operation, keeps the authorized payload identical to what is ultimately executed by the registry, and lets this specification remain agnostic to future RPP object and operation definitions.

9.1. DNS Update Transaction Type

The `rpp:transaction:dns-update` transaction type is used to update the DNS hosting of a domain name, by replacing the set of Host Data Objects referenced in the domain's `nameservers` property with a new set of Host Data Objects. The `objectId` property MUST contain the domain name of the target Domain Name Object.

The data property MUST contain a valid RPP JSON partial update request body (a JSON Patch document, as defined in the Partial Update rules of [\[I-D.ietf-rpp-json\]](#)), consisting of one `replace` operation for each existing Host Data Object being replaced. Each operation's path MUST be `/nameservers`, its `match` property MUST identify the existing Host Data Object being replaced by its `hostName`, and its `value` property MUST contain the new Host Data Object.

TODO

9.2. Transfer Transaction Type

The `rpp:transaction:transfer` transaction type is used to transfer the sponsorship of a domain to another registrar. The `objectId` property MUST contain the domain name of the target Domain Name Object, and the data property MUST contain a valid Transfer Process Object create request body, as defined in [\[I-D.ietf-rpp-json\]](#), for the transfer operation described in [\[I-D.ietf-rpp-core\]](#).

9.3. JSON Schema

This section provides normative JSON Schema definitions for the transaction types defined in this document. All schemas use JSON Schema draft 2020-12 [JSON-SCHEMA]

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$ref": "#/$defs/authorizeProcess.create",
  "$defs": {
    "authorizeProcess.create": {
      "type": "object",
      "properties": {
        "@type": { "type": "string", "const": "authorizeProcess" },
        "transactionType": { "type": "string", enum: ["rpp:transaction:dns-
update", "rpp:transaction:transfer"] },
        "id": { "type": "string" },
        "timestamp": { "type": "string", "format": "date-time" },
        "expiration": { "type": "string", "format": "date-time" },
        "objectId": { "type": "string" },
        "data": {
          "description": "The RPP JSON request body for the operation
identified by transactionType:
  a JSON object for a full resource representation, or a JSON Patch
array for a partial update,
  as defined in the RPP JSON Mapping specification."
        },
        "requestorName": { "type": "string" },
        "registrySig": { "type": "string" },
        "registryKeyId": { "type": "string" },
        "registrarSig": { "type": "string" },
        "registrarKeyId": { "type": "string" },
        "registrarApproval": { "type": "boolean" },
        "registrantApproval": { "type": "boolean" }
      },
      "required": ["@type", "transactionType"]
    }
  }
}
```

10. Examples

10.1. Key provisioning

Each party involved in the multi-party authorization flow MUST provide at least 1 public key at the registry.

TODO: Use the Organisation type?

```
{  
  ...  
}
```

10.2. Domain Update Request

Example 3rd party request to registry to replace the nameservers of a domain object with a new set of Host Data Objects, and replace the DNSSEC DS records. The data property contains an RPP JSON partial update (JSON Patch) request body as defined in [\[I-D.ietf-rpp-json\]](#), with two replace operations, one for replacing the existing nameservers and one for replacing the DNSSEC DS records. The example below shows a request to update the domain `test1.example`, replacing the existing nameservers with `ns1.dns.example` and `ns2.dns.example` and replacing the existing DS records with a new DS record with key tag 12345:

```
{
  "@type": "authorization",
  "transactionType": "rpp:transaction:dns-update",
  "timestamp": "2027-06-01T12:00:00Z",
  "expiration": "2027-06-01T12:10:00Z",
  "objectId": "test1.example",
  "data": [
    {
      "op": "replace",
      "path": "/nameservers",
      "value": [
        { "@type": "host", "hostName": "ns1.dns.example" },
        { "@type": "host", "hostName": "ns2.dns.example" }
      ]
    },
    {
      "op": "replace",
      "path": "/dns",
      "value": {
        "@type": "dnsData",
        "records": {
          "DS": [
            { "@type": "dnsRecord",
              "name": "test1.example",
              "ttl": 3600,
              "rdata": { "@type": "dsRdata",
                "keyTag": 12345,
                "algorithm": 8,
                "digestType": 2,
                "digest": "ABCDEF1234567890"
              }
            }
          ]
        }
      }
    }
  ]
}
```

TODO maybe split the example into two separate examples, one for replacing nameservers and one for replacing DS records, to make it clearer. and also create 2 separate transaction types, one for replacing nameservers and one for replacing DS records?

10.3. Domain Transfer Request

Example 3rd party request to registry to transfer the management of a domain object, signed by the 3rd party. The data property contains a Transfer Process Object create request body as defined in [\[I-D.ietf-rpp-json\]](#):

```
{
  "@type": "authorization",
  "transactionType": "rpp:transaction:transfer",
  "id": "TR-12345",
  "timestamp": "2027-06-01T12:00:00Z",
  "expiration": "2027-06-01T12:10:00Z",
  "objectId": "test1.example",
  "data": {
    "@type": "transferProcess",
    "transferDir": "pull"
  }
}
```

Example registry response to the 3rd party to transfer the management of a domain object, signed by the registry, this response is sent to the losing registrar to request the registrant's approval for the transfer:

The registry adds the following fields to the request: `id`, `requestorId`, `requestorName`, `registrySig`, and `registryKeyId`. The `id` is a unique identifier for the transaction, the `requestorName` is the public name of the requestor to be displayed in the consent screen, the `registrySig` is the signature of the registry, and the `registryKeyId` is the identifier of the public key used by the registry to sign the request.

```
{
  "@type": "authorization",
  "transactionType": "rpp:transaction:transfer",
  "id": "TR-12345",
  "timestamp": "2027-06-01T12:00:00Z",
  "expiration": "2027-06-01T12:10:00Z",
  "objectId": "test1.example",
  "requestorName": "Example Registrar",
  "registrySig": "xxxx",
  "registryKeyId": "xxxx"
}
```

Example response from losing registrar to the 3rd party to transfer the management of a domain object, signed by the registrar, the following fields are added to the request: `registrarSig`, `registrarKeyId`, `registrarApproval`, and `registrantApproval`. The `registrarSig` is the signature of the registrar, the `registrarKeyId` is the identifier of the public key used by the registrar to sign the request, and the `registrarApproval` and `registrantApproval` are boolean values indicating whether the registrar and registrant have approved the transfer.

This response is sent to the registry to request execution of the transfer:

```
{
  "@type": "authorization",
  "transactionType": "rpp:transaction:transfer",
  "id": "TR-12345",
  "timestamp": "2027-06-01T12:00:00Z",
  "expiration": "2027-06-01T12:10:00Z",
  "objectId": "test1.example",
  "registrySig": "xxxx",
  "registryKeyId": "xxxx",
  "registrarSig": "xxxx",
  "registrarKeyId": "xxxx",
  "registrarApproval": true,
  "registrantApproval": true
}
```

11. Security Considerations

The registrant's explicit consent is required for any RPP operation to be executed. The registrant's consent is obtained through the registrar's web-based consent screen, which is presented to the registrant by the registrar. The registrar **MUST** ensure that the registrant's consent is obtained before any RPP operation is executed.

The registry **MUST** ensure that the registrant is the owner of the domain name for which the RPP operation is requested, and that the registrant's consent is obtained before any RPP operation is executed.

What cryptographic algorithms and key management practices are used to sign and verify requests is outside the scope of this document, but it is **RECOMMENDED** that parties follow best practices for cryptographic security.

Signature verification **MUST** be performed locally by each verifying party using the public key of the signer, rather than by querying the signer's system at verification time. In particular, the registrar **MUST** verify the registry's signature (and the registry's verification of the 3rd party's signature) using the registry's public key, and **MUST NOT** rely on a live call back to the registry to confirm that a signature is valid. The registry already re-verifies all signatures, when the fully-signed request is submitted for execution (see [Section 4](#)).

This document does not specify any authorization and authentication mechanisms for the 3rd party, registry, and registrar to secure the HTTP endpoints used to exchange requests and responses. It is **RECOMMENDED** that parties follow best practices for securing HTTP endpoints, such as using TLS for end-to-end encryption, and OAuth 2.0 [[RFC6749](#)] for authorization.

Public keys provided by the 3rd party, registry, and registrar to the registry **SHOULD** be rotated periodically, and **MUST** be revoked if compromised.

TODO

12. IANA Considerations

IANA is requested to create a new registry for RPP multi-party transaction type identifiers, these identifiers are used to identify the type of RPP transaction used by 3rd parties to request authorization from the registry and registrar. The registry is to be called "RPP Transaction Types" and is to be maintained by the IETF RPP working group.

Name of the registry: RPP Multi-Party Transaction Types
Registry group: RESTful Provisioning Protocol (RPP)
Registration procedure: Expert Review

The following transaction types are defined in this document:

Identifier	Description
rpp:transaction:dns-update	Update DNS records for a domain
rpp:transaction:transfer	Transfer a domain to another registrar

Table 2: RPP multi-party authorization transaction types

TODO

13. Internationalization Considerations

TODO

14. Privacy Considerations

The registrant personally identifiable information (PII) is not included in the multi-party authorization flow, except for the registrant's explicit consent obtained through the registrar's web-based consent screen.

TODO

15. Change History

TODO

15.1. Version 00

- Created initial draft with core concepts and flows.

16. References

16.1. Normative References

[I-D.ietf-rpp-core] Wullink, M. and P. Kowalik, "RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-ietf-rpp-core-00, 6 July 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-rpp-core-00>>.

[I-D.ietf-rpp-data-objects] Kowalik, P. and M. Wullink, "RESTful Provisioning Protocol (RPP) Data Objects", Work in Progress, Internet-Draft, draft-ietf-rpp-data-objects-01, 3 July 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-rpp-data-objects-01>>.

- [I-D.ietf-rpp-json]** Wullink, M. and P. Kowalik, "JSON for RESTful Provisioning Protocol (RPP)", Work in Progress, Internet-Draft, draft-ietf-rpp-json-00, 6 July 2026, <<https://datatracker.ietf.org/doc/html/draft-ietf-rpp-json-00>>.
- [RFC2119]** Bradner, S., "Key words for use in RFCs to Indicate Requirement Levels", BCP 14, RFC 2119, DOI 10.17487/RFC2119, March 1997, <<https://www.rfc-editor.org/info/rfc2119>>.
- [RFC3986]** Berners-Lee, T., Fielding, R., and L. Masinter, "Uniform Resource Identifier (URI): Generic Syntax", STD 66, RFC 3986, DOI 10.17487/RFC3986, January 2005, <<https://www.rfc-editor.org/info/rfc3986>>.
- [RFC6749]** Hardt, D., Ed., "The OAuth 2.0 Authorization Framework", RFC 6749, DOI 10.17487/RFC6749, October 2012, <<https://www.rfc-editor.org/info/rfc6749>>.
- [RFC7517]** Jones, M., "JSON Web Key (JWK)", RFC 7517, DOI 10.17487/RFC7517, May 2015, <<https://www.rfc-editor.org/info/rfc7517>>.
- [RFC8785]** Rundgren, A., Jordan, B., and S. Erdtman, "JSON Canonicalization Scheme (JCS)", RFC 8785, DOI 10.17487/RFC8785, June 2020, <<https://www.rfc-editor.org/info/rfc8785>>.

16.2. Informative References

- [JSON-SCHEMA]** JSON Schema, "JSON Schema: A Media Type for Describing JSON Documents", 2020, <<https://json-schema.org/draft/2020-12/json-schema-core>>.

Acknowledgements

TODO

Authors' Addresses

Maarten Wullink

SIDN Labs

Email: maarten.wullink@sidn.nl

URI: <https://sidn.nl/>

Pawel Kowalik

DENIC

Email: pawel.kowalik@denic.de

URI: <https://denic.de/>